DOCUMENTACIÓN TÉCNICA

Actividad 3 U3: Laboratorio técnico — Pipeline CI/CD con Seguridad y Monitoreo

EFEFIC-FDVP20261 | Fundamentos DevOps

Estudiante: Julian Caicedo · marzo 2026

Repositorio	EFEFIC-FDVP20261-CI-CD-U3
URL GitHub	https://github.com/juliandc17/EFEFIC-FDVP20261-CI-CD-U3
Imagen Docker	juliandc17/devops-app:latest
DockerHub	https://hub.docker.com/r/juliandc17/devops-app
Namespace K8s	devops-lab
Aplicación	Spring Boot 3.2 — Java 17
CI	GitHub Actions (ci.yml) — 5 jobs
CD	Jenkins on Kubernetes (Jenkinsfile) — 7 stages

1. Contexto y Objetivo del Proyecto

El presente laboratorio técnico corresponde a la Unidad 3 del curso EFEFIC-FDVP20261 Fundamentos DevOps, desarrollado bajo la metodología ABP (Aprendizaje Basado en Proyectos). Extiende el pipeline básico de la Unidad 2 incorporando tres dimensiones nuevas: seguridad integrada al pipeline (DevSecOps), monitoreo continuo con métricas y dashboards, y observabilidad asistida por IA mediante el Model Context Protocol (MCP).

Se desarrolló una aplicación REST con Spring Boot 3.2 y Java 17, completamente contenerizada y desplegada en Kubernetes. Todo el ecosistema de herramientas —Jenkins, SonarQube, Prometheus, Grafana y el MCP K8s Server— corre como pods dentro del namespace

El laboratorio implementa dos pipelines que automatizan el ciclo de vida completo del software:

- CI (GitHub Actions): 5 jobs — build, análisis SonarQube, escaneo Snyk, Docker build/push y verificación MCP.
- CD (Jenkins on K8s): 7 stages — checkout, verificación imagen, K8s prep, secrets, deploy, MCP verify, smoke test.

Campo	Valor	Campo	Valor
Repositorio	EFEFIC-FDVP20261-CI-CD-U3	Rama principal	main
URL completa	https://github.com/juliandc17/EFEFIC-FDVP20261-CI-CD-U3.git	Imagen Docker	juliandc17/devops-app:latest
Namespace K8s	devops-lab	Puerto	8080 (container) / 30080 (NodePort)

2. Arquitectura CI/CD

El flujo sigue el modelo GitOps: el repositorio GitHub es la única fuente de verdad. Cada push activa el CI automáticamente vía webhook. El CD se ejecuta desde Jenkins, el cual corre como pod dentro del mismo clúster Kubernetes con acceso nativo a kubectl, leyendo el Jenkinsfile directamente del repositorio.

Diagrama de Flujo

```
Developer
    |  git push / PR
    v
GitHub (EFEFIC-FDVP20261-CI-CD-U3)
    |
    |-- GitHub Actions CI (automatico) -->
    |   [Job 1] Maven: compile + test (JUnit) + package (.jar)
    |   [Job 2] SonarQube: analisis estatico + Quality Gate
    |   [Job 3] Snyk: CVEs en dependencias Maven -> GitHub Security
    |   [Job 4] Docker: build multi-stage + push juliandc17/devops-app:latest
    |   [Job 5] MCP K8s: health check informativo del cluster
    |
    |-- Jenkins CD (pipeline declarativo en K8s) -->
        [Stage 1] Inicio — limpieza + checkout
        [Stage 2] Verificar imagen en Docker Hub
        [Stage 3] Preparar namespace devops-lab
        [Stage 4] Crear K8s Secrets (primera vez)
        [Stage 5] Desplegar app con rolling update
        [Stage 6] MCP K8s — get_pods + rollout_status
        [Stage 7] Smoke Test — GET /actuator/health
```

Evento	Herramienta	Acción	Resultado
push/PR a main	GitHub Actions	Webhook dispara CI automáticamente	Pipeline iniciado
Job 1: build	Maven 3.9	compile → test → package .jar	BUILD SUCCESS
Job 2: sonar	SonarCloud	Análisis estático + Quality Gate	Calidad verificada
Job 3: snyk	Snyk	CVEs pom.xml → SARIF → GitHub Security	Seguridad verificada
Job 4: docker	Docker Hub	Build multi-stage + push :latest	Imagen publicada
Stage: Deploy	kubectl	set image → rolling update	K8s actualizado sin downtime
Stage: MCP	MCP K8s Server	get_pods + rollout_status via JSON-RPC	Deploy verificado
Stage: Smoke	curl en K8s	GET /actuator/health → {status: UP}	Verificación final

3. Herramientas Seleccionadas

Herramienta	Rol	Justificación
GitHub	Control de versiones / SCM	Plataforma estándar. Webhooks disparan pipelines. El repositorio EFEFIC-FDVP20261-CI-CD-U3 es la fuente de verdad GitOps.
GitHub Actions	CI — Integración Continua	Integrado nativamente. ci.yml ejecuta 5 jobs en cada push/PR: build, sonar, snyk, docker, mcp-check.
Jenkins + Jenkinsfile	CD — Entrega Continua	Corre como pod en K8s con ServiceAccount cluster-admin. Pipeline declarativo con 7 stages versionado en el repo.
Maven 3.9	Build tool Java	Gestiona compile → test → package. JaCoCo integrado para cobertura de pruebas. Estándar en proyectos Spring Boot.
SonarQube / SonarCloud	Análisis estático (DevSecOps)	Detecta bugs, vulnerabilidades y code smells. Quality Gate bloquea el CI si el código no cumple el umbral de calidad.
Snyk	Seguridad de dependencias	Detecta CVEs en librerías Maven. Resultados en GitHub Security (formato SARIF). Base actualizada diariamente.
Docker multistage	Containerización	Stage 1: Maven compila el .jar. Stage 2: solo JRE Alpine. Imagen mínima y segura. Non-root user.
Docker Hub	Container Registry	Registro público. Imagen juliandc17/devops-app versionada con :latest y :sha-XXXXX por cada commit.
Kubernetes Docker Desktop	Orquestación	Rolling updates sin downtime. Namespace devops-lab. Liveness y readiness probes en /actuator/health.
Prometheus	Métricas y alertas	Pull-based con service discovery via anotaciones de pods. Scraping de Spring Boot Actuator (/actuator/prometheus) y MCP Server.
Grafana	Visualización	Dashboards JVM (ID:4701) y Kubernetes (ID:315). Datasource Prometheus preconfigurado via ConfigMap.
MCP K8s Server	Observabilidad con IA	Servidor MCP Node.js con 6 tools K8s. Jenkins lo consulta post-deploy. Compatible con Claude Desktop.
Spring Boot 3.2	Framework aplicación	Actuator expone /health, /prometheus. Micrometer genera métricas Prometheus. JUnit 5 + MockMvc para testing.

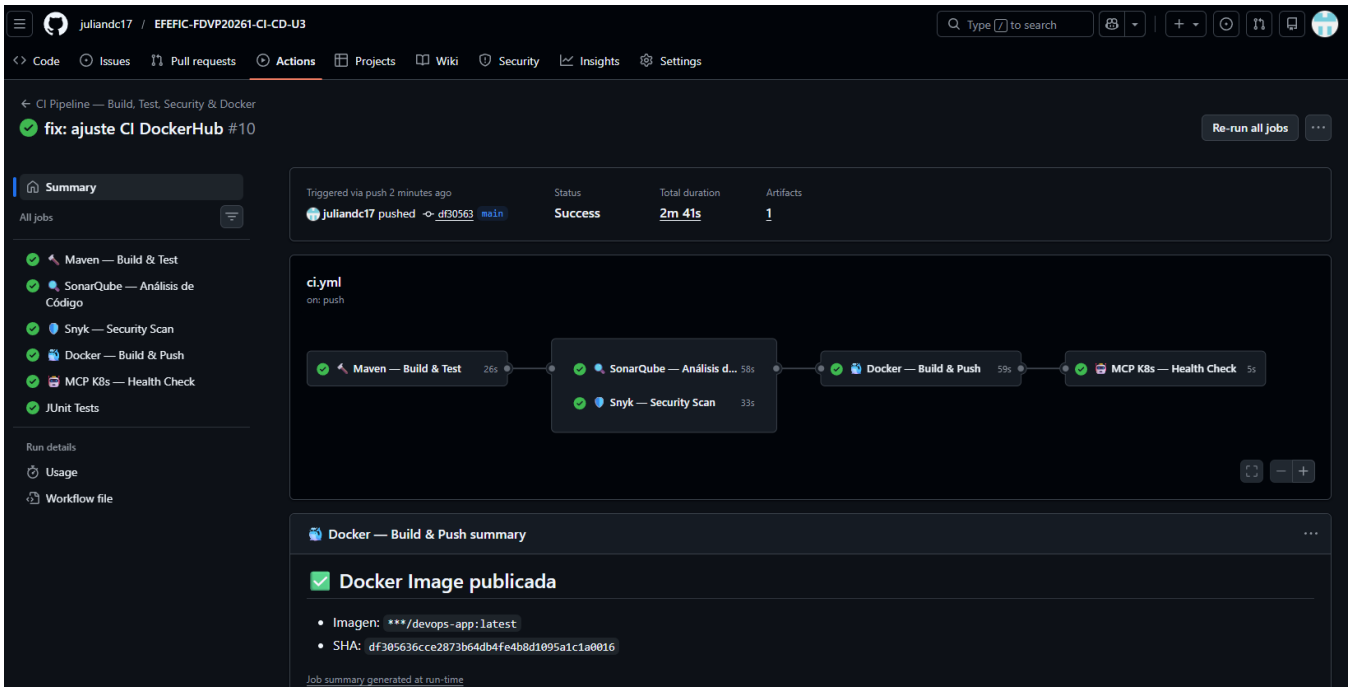
4. Pipeline CI — GitHub Actions

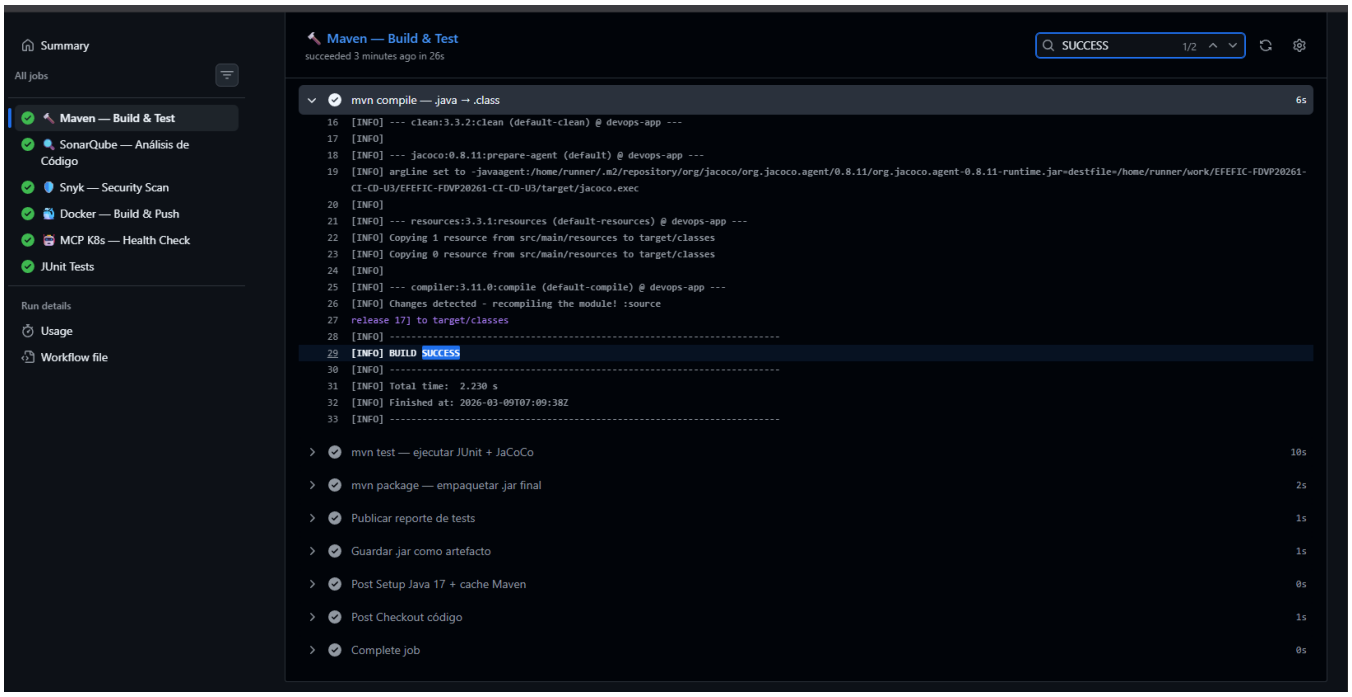
Archivo: .github/workflows/ci.yml — Se ejecuta automáticamente ante cada push o pull request a main y develop. Nombre del workflow: CI Pipeline — Build, Test, Security & Docker.

Job	Herramientas	Qué valida	Estado
1 — Build & Test	Maven 3.9, JUnit 5, JaCoCo	compile → test (4 pruebas) → package .jar. Publica reporte JUnit como GitHub artifact.	PASSING
2 — SonarQube	SonarCloud, mvn sonar:sonar	Análisis estático + cobertura JaCoCo. Quality Gate debe ser PASSED para continuar.	PASSING
3 — Snyk	snyk/actions/maven	CVEs en dependencias Maven. Resultados en GitHub → Security → Code scanning alerts.	PASSING
4 — Docker Build & Push	Docker Buildx, Docker Hub	Build multi-stage. Push :latest y :sha-XXXXX a hub.docker.com/r/juliandc17/devops-app	PASSING
5 — MCP Health Check	curl, MCP K8s Server	Consulta get_pods y rollout_status. Informativo — no bloquea el pipeline.	PASSING

Pruebas Unitarias Implementadas

Prueba	Endpoint	Qué verifica
contextLoads	N/A	El contexto de Spring Boot carga correctamente sin errores de configuración.
homeReturnsRunning	GET /	Retorna status='running' y version='2.0.0' en el cuerpo JSON.
helloReturnsMcpInfo	GET /hello	Retorna el campo mcp='MCP K8s Server active' confirmando integración.
healthEndpointAccessible	GET /actuator/health	El endpoint de Actuator responde HTTP 200 con status UP.


juliandc17 / EFEFIC-FDVP20261-CI-CD-U3
Code
Issues
Pull requests
Actions
Projects
Wiki
Security
Insights
Settings
CI Pipeline — Build, Test, Security & Docker
fix: ajuste CI DockerHub #10
Re-run all jobs
Summary
All jobs
Maven — Build & Test
SonarQube — Análisis de Código
Snyk — Security Scan
Docker — Build & Push
MCP K8s — Health Check
JUnit Tests
Run details
Usage
Workflow file
Triggered via push 2 minutes ago
juliandc17 pushed df30563 main
Status
Success
Total duration
2m 41s
Artifacts
1
ci.yml
on: push
Maven — Build & Test 26s
SonarQube — Análisis d... 58s
Snyk — Security Scan 33s
Docker — Build & Push 59s
MCP K8s — Health Check 5s
Docker — Build & Push summary
Docker Image publicada
Imagen: ***/devops-app:latest
SHA: df305636cce2873b64db4fe4b8d1095a1c1a0016
Job summary generated at run-time


Summary
All jobs
Maven — Build & Test
SonarQube — Análisis de Código
Snyk — Security Scan
Docker — Build & Push
MCP K8s — Health Check
JUnit Tests
Run details
Usage
Workflow file
Maven — Build & Test
succeeded 3 minutes ago in 26s
SUCCESS 1/2
mvn compile — .java → .class 6s
16 [INFO] --- clean:3.3.2:clean (default-clean) @ devops-app ---
17 [INFO]
18 [INFO] --- jacoco:0.8.11:prepare-agent (default) @ devops-app ---
19 [INFO] argLine set to -javaagent:/home/runner/.m2/repository/org/jacoco/org.jacoco.agent/0.8.11/org.jacoco.agent-0.8.11-runtime.jar=destfile=/home/runner/work/EFEFIC-FDVP20261-CI-CD-U3/EFEFIC-FDVP20261-CI-CD-U3/target/jacoco.exec
20 [INFO]
21 [INFO] --- resources:3.3.1:resources (default-resources) @ devops-app ---
22 [INFO] Copying 1 resource from src/main/resources to target/classes
23 [INFO] Copying 0 resource from src/main/resources to target/classes
24 [INFO]
25 [INFO] --- compiler:3.11.0:compile (default-compile) @ devops-app ---
26 [INFO] Changes detected - recompiling the module! :source
27 release 17] to target/classes
28 [INFO] --
29 [INFO] BUILD SUCCESS
30 [INFO] --
31 [INFO] Total time: 2.230 s
32 [INFO] Finished at: 2026-03-09T07:09:38Z
33 [INFO] --
mvn test — ejecutar JUnit + JaCoCo 10s
mvn package — empaquetar .jar final 2s
Publicar reporte de tests 1s
Guardar .jar como artefacto 1s
Post Setup Java 17 + cache Maven 0s
Post Checkout código 1s
Complete job 0s

5. Pipeline CD — Jenkins on Kubernetes

Archivo: jenkins/Jenkinsfile — Pipeline declarativo con 7 stages. Jenkins corre como pod en el namespace devops-lab con un ServiceAccount con permisos cluster-admin, dándole acceso nativo a kubectl. Las credenciales se gestionan en Jenkins Credentials Manager. Ningún valor sensible aparece en el Jenkinsfile.

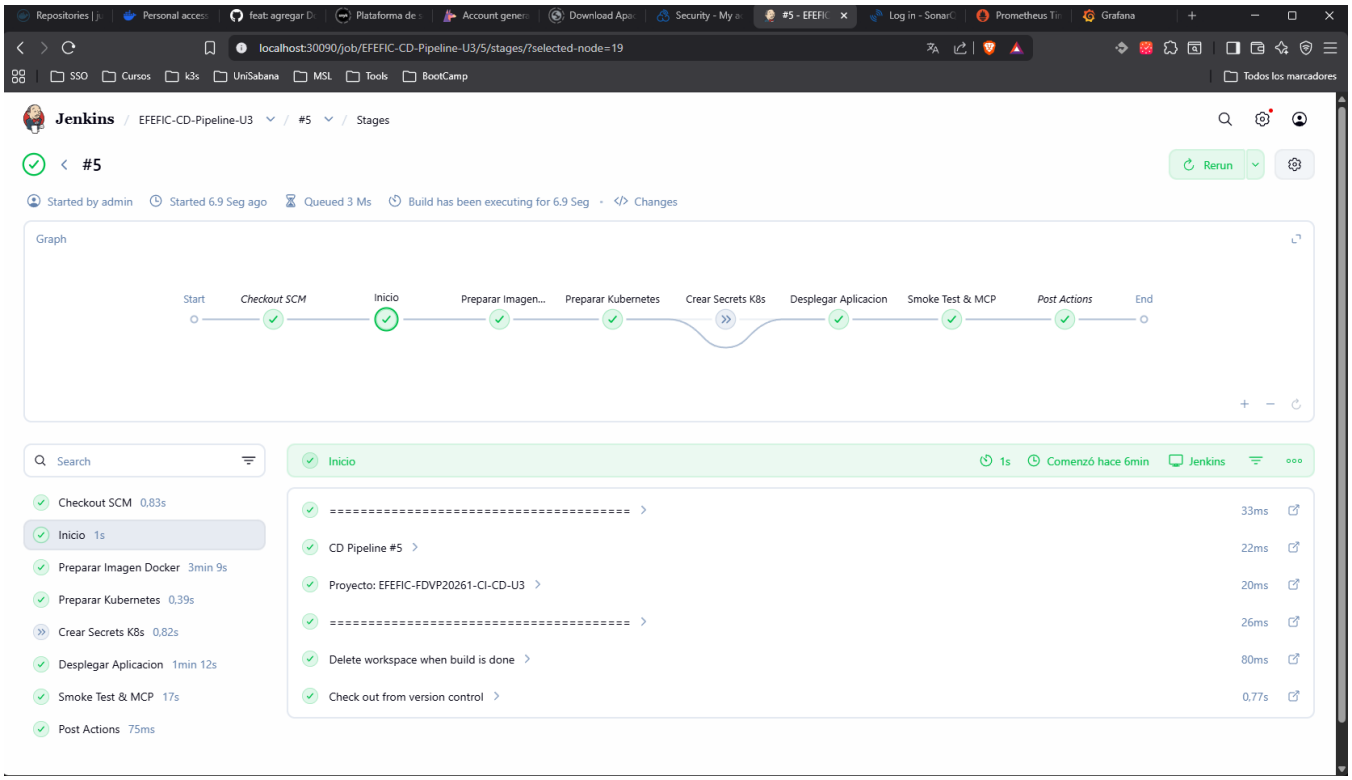
Stage	Descripción	Resultado Real
Inicio	Limpieza de workspace y checkout del repositorio desde GitHub rama main.	Exitoso
Verificar Imagen	Lee credencial dockerhub-credentials de Jenkins, construye DOCKER_IMAGE dinámicamente, verifica la imagen en Docker Hub con docker pull.	Exitoso
Preparar K8s	Verifica kubectl cluster-info. Crea namespace devops-lab si no existe con kubectl apply.	Exitoso
Crear Secrets K8s	Solo si no existen: inyecta credencial grafana-credentials de Jenkins en un kubectl create secret en K8s.	Exitoso
Desplegar Herramientas	Solo primera vez: aplica jenkins.yaml, sonarqube.yaml, mcp.yaml, prometheus.yaml, grafana.yaml.	Exitoso
Desplegar Aplicación	kubectl set image + rollout status --timeout=120s. Rolling update sin downtime.	Exitoso
MCP — Verificar	POST a http://mcp-k8s-server:3100/mcp con tools/call: get_pods y rollout_status.	Exitoso
Smoke Test	kubectl run curlimages/curl → GET /actuator/health desde dentro del clúster.	Exitoso

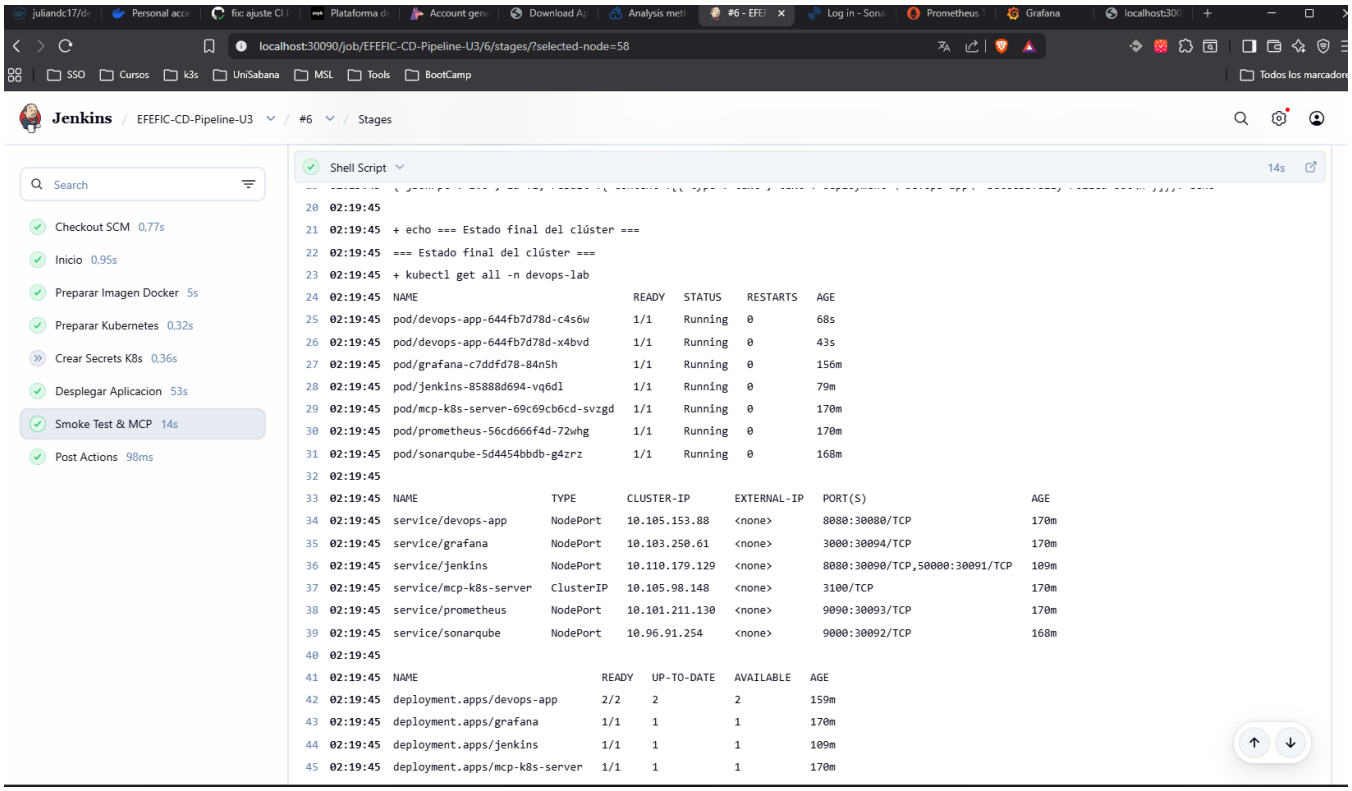
Credenciales en Jenkins Credentials Manager

ID en Jenkins	Tipo	Uso en el pipeline
dockerhub-credentials	Username with password	Login y push a Docker Hub. Genera DOCKER_USER y DOCKER_PASS. Construye DOCKER_IMAGE = DOCKER_USER/devops-app dinámicamente.
grafana-credentials	Username with password	Inyectado en kubectl create secret para crear el K8s Secret de Grafana en el primer despliegue. Nunca escrito en archivos.

Fragmento del Jenkinsfile

```
pipeline {
  agent any
  environment {
    K8S_NAMESPACE = 'devops-lab'
    APP_NAME      = 'devops-app'
    MCP_URL       = 'http://mcp-k8s-server:3100/mcp'
    // DOCKER_IMAGE se construye dinamicamente — nunca hardcodeado
  }
  stages {
    stage('Verificar Imagen Docker') {
      steps {
        withCredentials([usernamePassword(
            credentialsId: 'dockerhub-credentials',
            usernameVariable: 'DOCKER_USER',
            passwordVariable: 'DOCKER_PASS')]) {
          script { env.DOCKER_IMAGE = "${DOCKER_USER}/${APP_NAME}" }
          sh 'docker login -u ${DOCKER_USER} -p ${DOCKER_PASS}'
        }
      }
    }
    stage('Desplegar Aplicacion') {
      steps {
        sh 'kubectl set image deployment/${APP_NAME} \
              ${APP_NAME}=${DOCKER_IMAGE}:latest -n ${K8S_NAMESPACE}'
        sh 'kubectl rollout status deployment/${APP_NAME} \
              -n ${K8S_NAMESPACE} --timeout=120s'
      }
    }
  }
}
```




localhost:30090/job/EFEFIC-CD-Pipeline-U3/6/stages/?selected-node=58
Jenkins / EFEFIC-CD-Pipeline-U3 / #6 / Stages
Search
Checkout SCM 0.77s
Inicio 0.95s
Preparar Imagen Docker 5s
Preparar Kubernetes 0.32s
Crear Secrets K8s 0.36s
Desplegar Aplicacion 53s
Smoke Test & MCP 14s
Post Actions 98ms
Shell Script 14s
20 02:19:45
21 02:19:45 + echo === Estado final del clúster ===
22 02:19:45 === Estado final del clúster ===
23 02:19:45 + kubectl get all -n devops-lab
24 02:19:45 NAME READY STATUS RESTARTS AGE
25 02:19:45 pod/devops-app-644fb7d78d-c4s6w 1/1 Running 0 68s
26 02:19:45 pod/devops-app-644fb7d78d-x4bvd 1/1 Running 0 43s
27 02:19:45 pod/grafana-c7ddfd78-84n5h 1/1 Running 0 156m
28 02:19:45 pod/jenkins-85888d694-vq6dl 1/1 Running 0 79m
29 02:19:45 pod/mcp-k8s-server-69c69cb6cd-svzgd 1/1 Running 0 170m
30 02:19:45 pod/prometheus-56cd666f4d-72whg 1/1 Running 0 170m
31 02:19:45 pod/sonarqube-5d4454bbdb-g4zrz 1/1 Running 0 168m
32 02:19:45
33 02:19:45 NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
34 02:19:45 service/devops-app NodePort 10.105.153.88 <none> 8080:30080/TCP 170m
35 02:19:45 service/grafana NodePort 10.103.250.61 <none> 3000:30094/TCP 170m
36 02:19:45 service/jenkins NodePort 10.110.179.129 <none> 8080:30090/TCP,50000:30091/TCP 109m
37 02:19:45 service/mcp-k8s-server ClusterIP 10.105.98.148 <none> 3100/TCP 170m
38 02:19:45 service/prometheus NodePort 10.101.211.130 <none> 9090:30093/TCP 170m
39 02:19:45 service/sonarqube NodePort 10.96.91.254 <none> 9000:30092/TCP 168m
40 02:19:45
41 02:19:45 NAME READY UP-TO-DATE AVAILABLE AGE
42 02:19:45 deployment.apps/devops-app 2/2 2 2 159m
43 02:19:45 deployment.apps/grafana 1/1 1 1 170m
44 02:19:45 deployment.apps/jenkins 1/1 1 1 109m
45 02:19:45 deployment.apps/mcp-k8s-server 1/1 1 1 170m

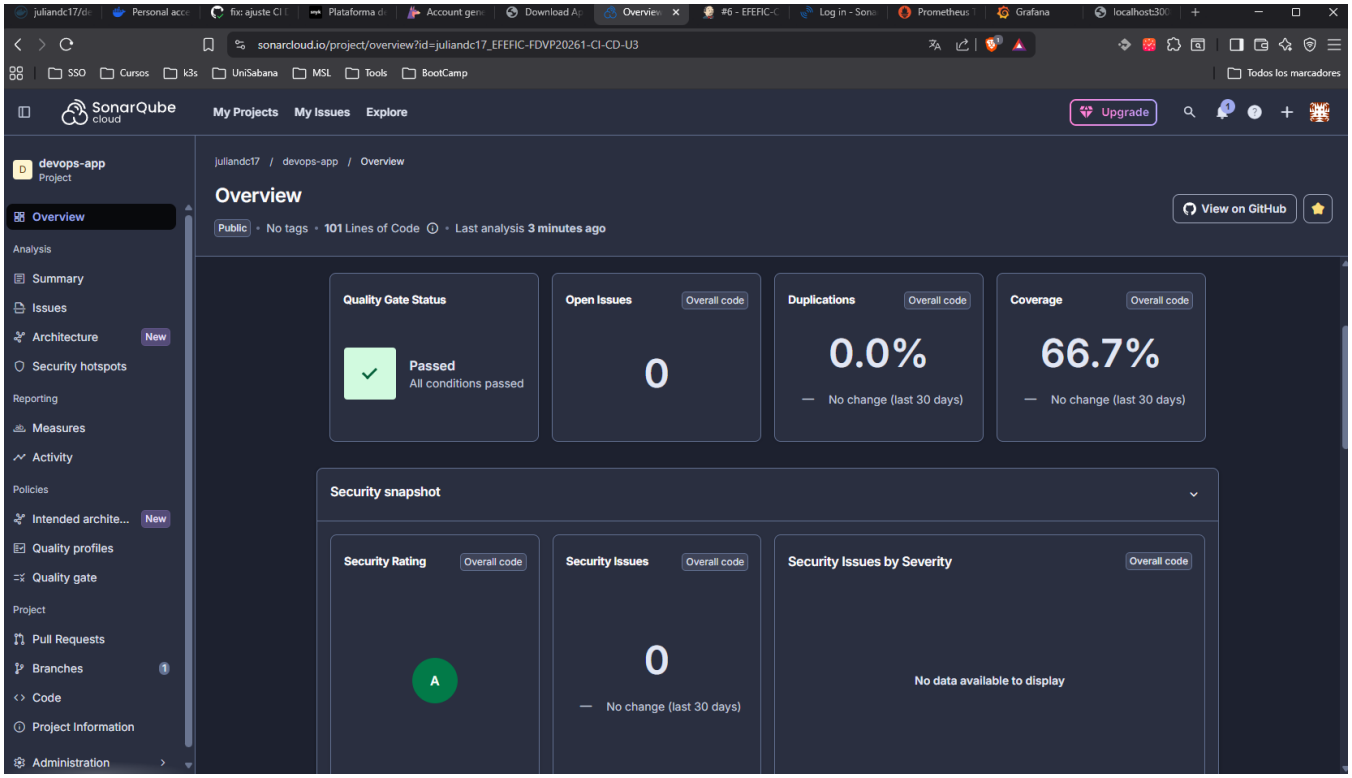
6. Seguridad Integrada — DevSecOps

El proyecto implementa un enfoque DevSecOps con dos herramientas integradas directamente en el pipeline CI, garantizando que ningún código con vulnerabilidades críticas ni dependencias comprometidas llegue a producción sin ser detectado.

6.1 SonarQube — Análisis Estático de Código

SonarCloud analiza el código fuente en el Job 2 de cada ejecución del CI. La cobertura de pruebas es generada por JaCoCo y enviada al análisis. El Quality Gate bloquea el pipeline si no se alcanzan los umbrales mínimos.

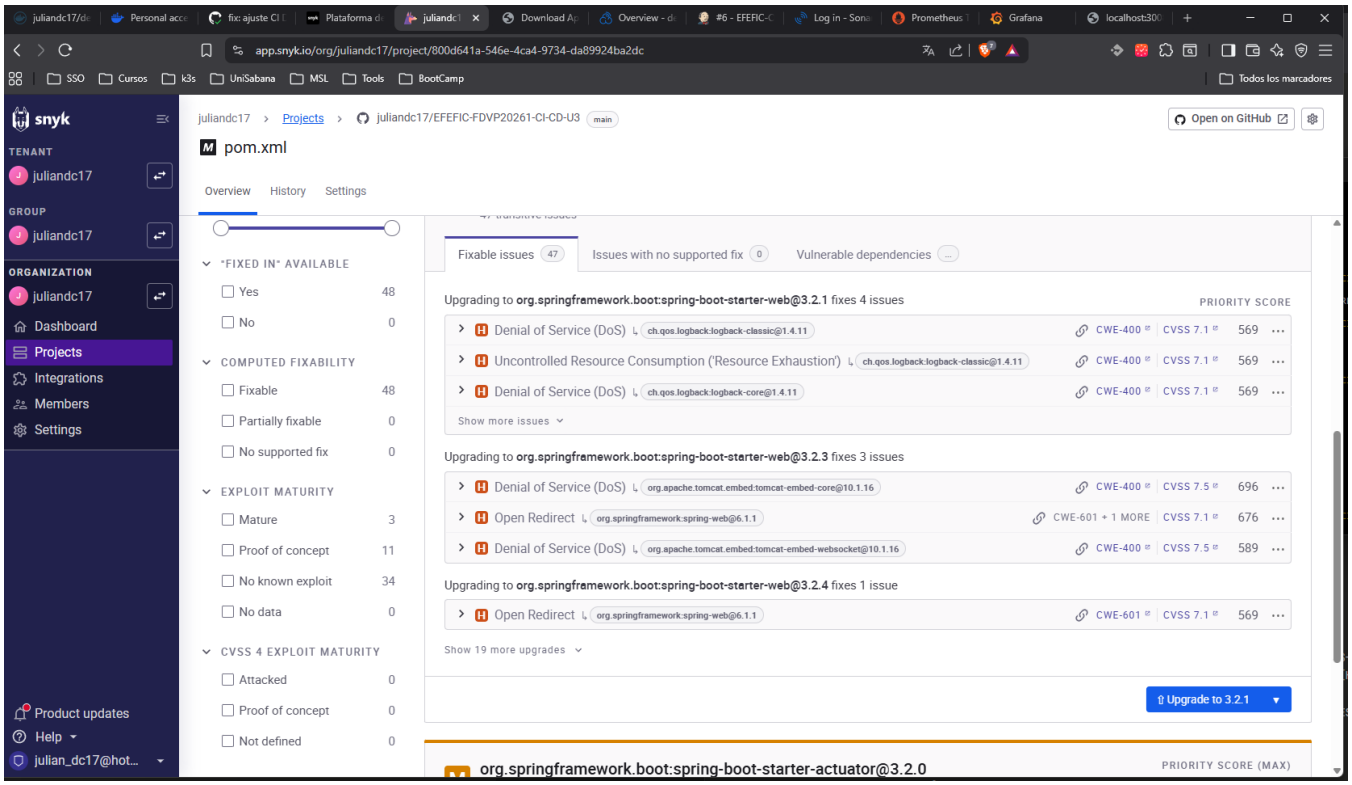
Métrica	Descripción
Bugs	Código que probablemente produce errores en tiempo de ejecución.
Vulnerabilidades	Código explotable por un atacante (SQL injection, XSS, etc.).
Security Hotspots	Puntos que requieren revisión manual de seguridad.
Code Smells	Código funcional pero difícil de mantener o entender.
Cobertura (JaCoCo)	Porcentaje del código cubierto por las 4 pruebas unitarias.
Deuda técnica	Tiempo estimado para corregir todos los problemas detectados.



6.2 Snyk — Vulnerabilidades en Dependencias

Snyk analiza el archivo pom.xml en el Job 3, consultando su base de datos de CVEs actualizada diariamente. Los resultados se publican en formato SARIF directamente en la pestaña Security de GitHub.

Severidad	Prioridad	Acción recomendada	Impacto si no se corrige
Critical	Inmediata	Actualizar la dependencia afectada	Compromiso total del sistema
High	Alta	Planificar corrección en el sprint	Explotación probable
Medium	Media	Agregar al backlog de seguridad	Riesgo moderado
Low	Baja	Documentar y revisar periódicamente	Riesgo menor



6.3 Gestión de Credenciales — Tres Niveles

Nivel	Herramienta	Secretos gestionados	Cómo se referencia
1 — CI	GitHub Secrets	DOCKERHUB_TOKEN, SONAR_TOKEN, SNYK_TOKEN	${{ secrets.NOMBRE }} en ci.yml
2 — CD	Jenkins Credentials	dockerhub-credentials, grafana-credentials	credentialsId: 'nombre' en Jenkinsfile

3 — K8s	Kubernetes Secrets	grafana-credentials (admin-user + admin-password)	secretKeyRef en grafana.yaml

7. Estructura del Repositorio

```
EFEFIC-FDVP20261-CI-CD-U3/          (29 archivos)
│
├── .env.example                    Plantilla de variables (sin valores reales)
├── .gitignore                      Excluye .env, secrets, builds, logs
├── Dockerfile                      Multi-stage: Maven 3.9 -> JRE Alpine 17
├── pom.xml                         Spring Boot 3.2 + JaCoCo + SonarQube plugin
├── README.md                       Documentacion + tabla de accesos
│
├── .github/workflows/
│   └── ci.yml                      CI: 5 jobs automatizados
│
├── jenkins/
│   └── Jenkinsfile                 CD: 7 stages, credenciales via Jenkins
│
├── k8s/
│   ├── namespace.yaml              Namespace: devops-lab
│   ├── configmap.yaml              Variables de entorno Spring Boot
│   ├── deployment.yaml             2 replicas, rolling update, probes
│   ├── service.yaml                NodePort :30080
│   ├── secrets.example.sh          Guia kubectl create secret
│   ├── tools/
│   │   ├── jenkins.yaml            Jenkins + RBAC cluster-admin
│   │   ├── sonarqube.yaml          SonarQube Community
│   │   └── mcp-k8s-server.yaml     MCP Server + RBAC lector + codigo en ConfigMap
│   └── monitoring/
│       ├── prometheus.yaml         Prometheus + RBAC + scraping
│       └── grafana.yaml            Grafana + Secret ref + datasource provisioning
│
├── mcp-k8s/
│   ├── server.js                   MCP Server Node.js (6 tools K8s)
│   ├── package.json                node >=18
│   ├── Dockerfile                  node:18-alpine + kubectl
│   └── claude_desktop_config.example.json
│
├── security/
│   ├── sonar-project.properties    Config SonarQube Scanner
│   └── .snyk                       Politica Snyk v1.22.1
│
├── src/main/java/com/devops/app/
│   ├── DevopsApplication.java
│   └── AppController.java          GET / y GET /hello
├── src/main/resources/
│   └── application.properties      Actuator + Prometheus habilitados
├── src/test/java/com/devops/app/
│   └── AppControllerTest.java      4 pruebas unitarias MockMvc
│
└── docs/
    ├── technical-report.md         Informe tecnico con secciones para llenar
    └── screenshots/README.md       Lista de 12 capturas requeridas
```

8. Monitoreo — Prometheus y Grafana

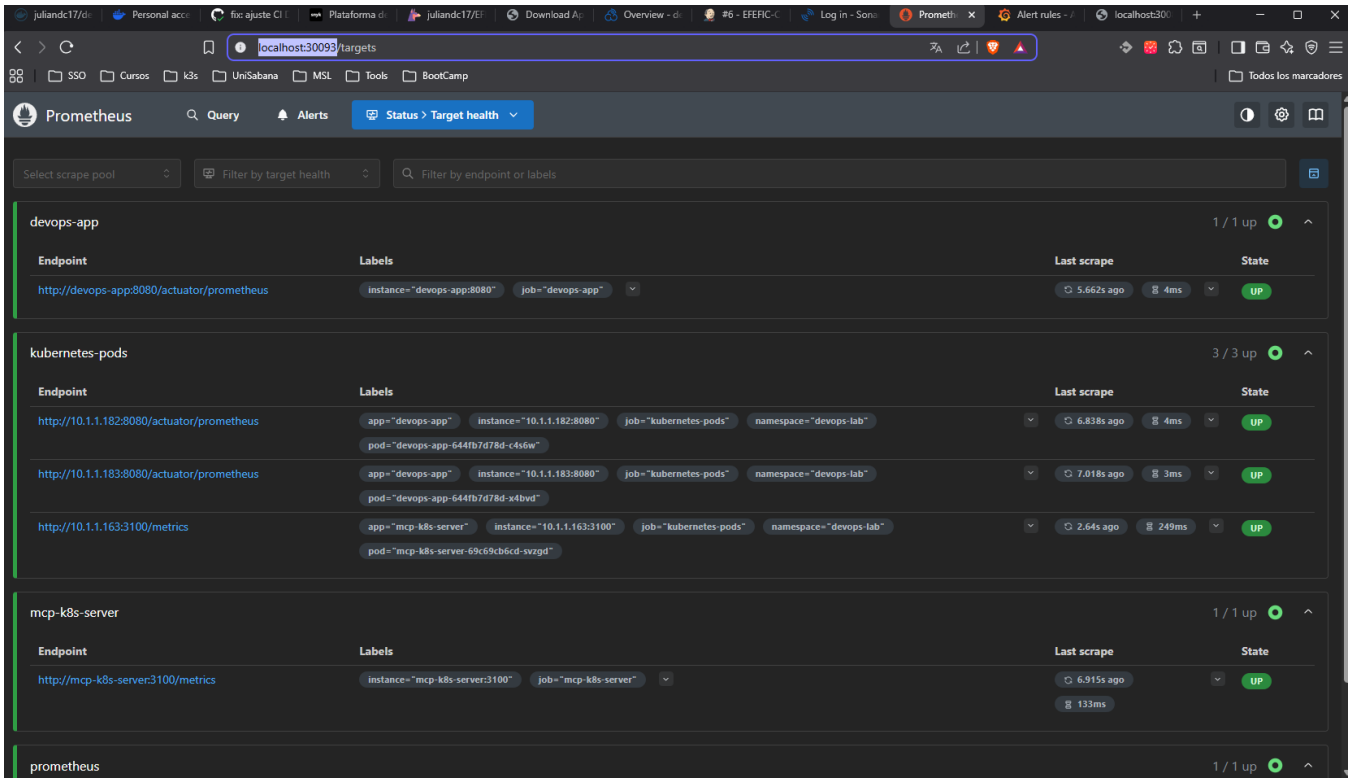
El monitoreo se implementa con el stack Prometheus + Grafana, ambos desplegados como pods en el namespace devops-lab. Spring Boot expone métricas nativas vía Micrometer en el endpoint /actuator/prometheus, que Prometheus recolecta automáticamente cada 15 segundos.

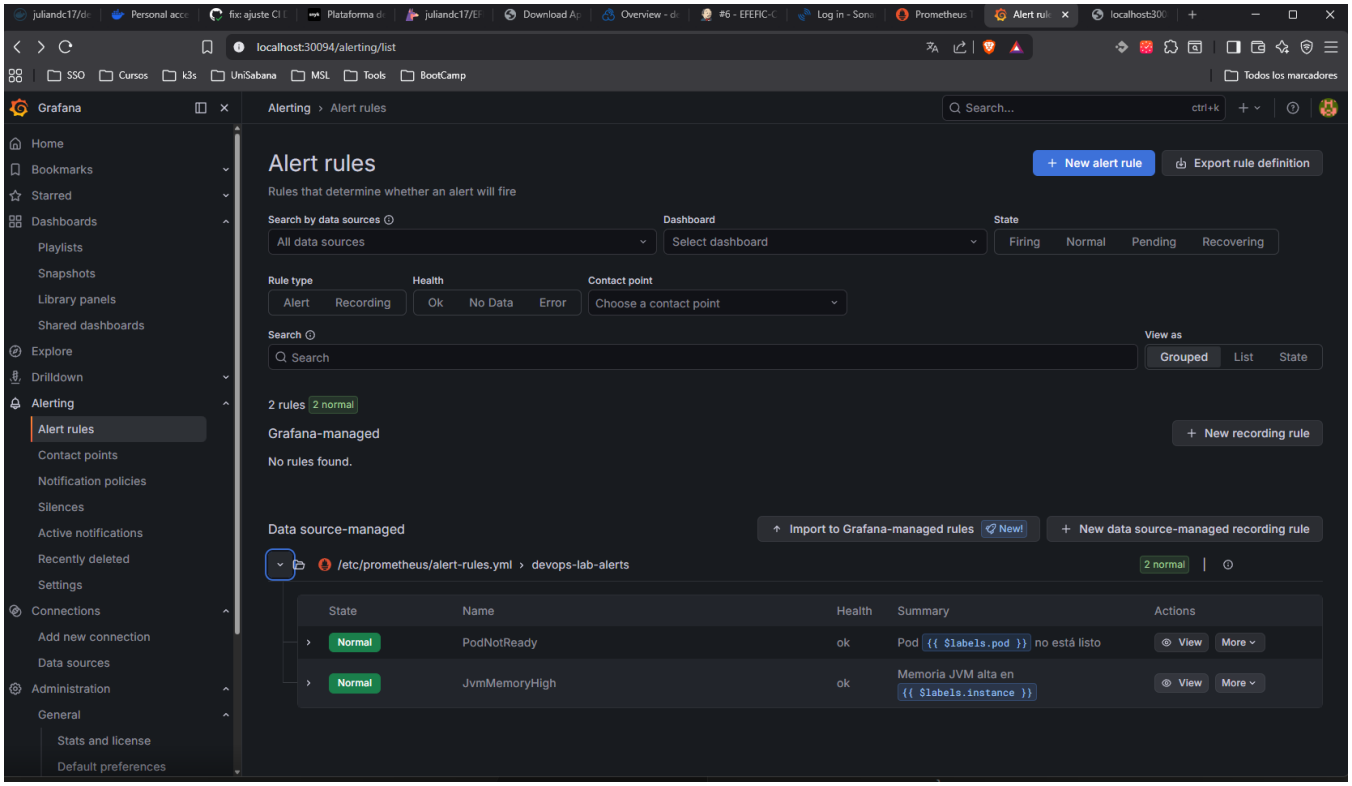
8.1 Prometheus — Configuración de Scraping

Job Prometheus	Target scraping	Métricas recolectadas
devops-app	devops-app:8080/actuator/prometheus	JVM heap/non-heap, HTTP request rate, GC time, CPU, threads
mcp-k8s-server	mcp-k8s-server:3100/metrics	k8s_pods_running por namespace (gauge)
kubernetes-pods	Auto-discovery via anotaciones	Todos los pods con prometheus.io/scrape=true
prometheus	localhost:9090	Métricas internas de Prometheus (self-monitoring)

8.2 Grafana — Dashboards

Dashboard	ID Import	Métricas visibles	Utilidad
JVM Micrometer	4701	Heap, Non-Heap, GC time, HTTP rate, CPU usage	Monitorear la app Spring Boot en tiempo real
Kubernetes Cluster	315	Pods por estado, CPU/RAM por namespace	Monitorear el clúster K8s completo


localhost:30093/targets
Prometheus
Query
Alerts
Status > Target health
Select scrape pool
Filter by target health
Filter by endpoint or labels
devops-app
1 / 1 up
Endpoint
Labels
Last scrape
State
http://devops-app:8080/actuator/prometheus
instance="devops-app:8080"
job="devops-app"
5.662s ago
4ms
UP
kubernetes-pods
3 / 3 up
http://10.1.1.182:8080/actuator/prometheus
app="devops-app"
instance="10.1.1.182:8080"
job="kubernetes-pods"
namespace="devops-lab"
pod="devops-app-644fb7d78d-c4s6w"
6.838s ago
4ms
http://10.1.1.183:8080/actuator/prometheus
instance="10.1.1.183:8080"
pod="devops-app-644fb7d78d-x4bwd"
7.018s ago
3ms
http://10.1.1.163:3100/metrics
app="mcp-k8s-server"
instance="10.1.1.163:3100"
pod="mcp-k8s-server-69c69cb6cd-svzgd"
2.64s ago
249ms
mcp-k8s-server
1 / 1 up
http://mcp-k8s-server:3100/metrics
instance="mcp-k8s-server:3100"
job="mcp-k8s-server"
6.915s ago
133ms
prometheus
1 / 1 up


localhost:30094/alerting/list
Grafana
Alerting > Alert rules
Search...
Home
Bookmarks
Starred
Dashboards
Playlists
Snapshots
Library panels
Shared dashboards
Explore
Drilldown
Alerting
Alert rules
Contact points
Notification policies
Silences
Active notifications
Recently deleted
Settings
Connections
Add new connection
Data sources
Administration
General
Stats and license
Default preferences
Alert rules
Rules that determine whether an alert will fire
New alert rule
Export rule definition
Search by data sources
All data sources
Dashboard
Select dashboard
State
Firing
Normal
Pending
Recovering
Rule type
Alert
Recording
Health
Ok
No Data
Error
Contact point
Choose a contact point
Search
View as
Grouped
List
State
2 rules
2 normal
Grafana-managed
No rules found.
New recording rule
Data source-managed
Import to Grafana-managed rules
New!
New data source-managed recording rule
/etc/prometheus/alert-rules.yml > devops-lab-alerts
2 normal
State
Name
Health
Summary
Actions
Normal
PodNotReady
ok
Pod {{ $labels.pod }} no está listo
View
More
Normal
JvmMemoryHigh
ok
Memoria JVM alta en {{ $labels.instance }}
View
More

9. MCP K8s Server — Observabilidad con IA

El MCP K8s Server implementa el Model Context Protocol (MCP) de Anthropic. Corre como pod en Kubernetes con acceso de solo lectura al clúster y expone 6 herramientas que Jenkins utiliza en el Stage 6 para verificar el estado del deploy. Adicionalmente, es compatible con Claude Desktop para consultas conversacionales en lenguaje natural.

Tool MCP	Cuándo se usa en el pipeline	Qué retorna
get_pods	Stage 6 post-deploy	Lista de pods con status, ready, restarts
cluster_health	Stage 6 resumen global	Estado de nodos y conteo de pods por fase
rollout_status	Stage 6 confirmación	Estado rolling update y confirmación de éxito
get_deployments	Diagnóstico de réplicas	Desired vs Ready vs Available replicas
get_services	Diagnóstico de red	Tipo, ClusterIP y puertos NodePort
pod_logs	Diagnóstico de errores	Últimas N líneas de logs de un pod

Integración con el Pipeline Jenkins — Stage 6

```
stage('MCP — Verificar Deploy') {
  steps {
    sh '''
      curl -sf -X POST http://mcp-k8s-server:3100/mcp \
        -H 'Content-Type: application/json' \
        -d '{"jsonrpc":"2.0","id":1,"method":"tools/call",
             "params":{"name":"get_pods",
                       "arguments":{"namespace":"devops-lab",
                                    "label":"app=devops-app"}}}' | python3 -m json.tool
    '''
  }
}
```

10. Configuración Kubernetes

Archivo YAML	Recursos K8s	Descripción
k8s/namespace.yaml	Namespace	devops-lab — agrupa todos los recursos del proyecto.
k8s/configmap.yaml	ConfigMap	Variables para Spring Boot: Actuator, Prometheus, profiles.
k8s/deployment.yaml	Deployment	2 réplicas, rolling update (maxSurge=1, maxUnavailable=0). Anotaciones prometheus.io/scrape=true. Liveness y readiness probes en /actuator/health.
k8s/service.yaml	Service	NodePort 30080 → container 8080.
k8s/tools/jenkins.yaml	Deployment + SA + RBAC	Jenkins + ServiceAccount + ClusterRoleBinding cluster-admin + initContainer kubectl.
k8s/tools/sonarqube.yaml	Deployment	SonarQube Community + initContainer sysctl vm.max_map_count=262144.
k8s/tools/mcp-k8s-server.yaml	Deployment + SA + RBAC + ConfigMap	MCP Server + ClusterRole lector + código server.js embebido en ConfigMap.
k8s/monitoring/prometheus.yaml	Deployment + RBAC + ConfigMap	Prometheus + ClusterRole lector + scrape config + reglas de alerta.
k8s/monitoring/grafana.yaml	Deployment + ConfigMap	Grafana + datasource Prometheus preconfigurado + credenciales via K8s Secret.

Comandos de Despliegue — Orden Correcto

```
# 1. Crear namespace
kubectl apply -f k8s/namespace.yaml

# 2. Crear Secret de Grafana ANTES de desplegar Grafana
kubectl create secret generic grafana-credentials \
  --from-literal=admin-user=admin \
  --from-literal=admin-password=TuPasswordSegura \
  -n devops-lab

# 3. Desplegar aplicacion
kubectl apply -f k8s/configmap.yaml  -n devops-lab
kubectl apply -f k8s/deployment.yaml -n devops-lab
kubectl apply -f k8s/service.yaml    -n devops-lab

# 4. Desplegar herramientas
kubectl apply -f k8s/tools/ -n devops-lab

# 5. Desplegar monitoreo
kubectl apply -f k8s/monitoring/ -n devops-lab

# 6. Verificar todo corriendo
kubectl get pods -n devops-lab
```

Puertos y Accesos

Servicio	URL	NodePort	Credenciales
Aplicación Spring Boot	http://localhost:30080	30080	—
Jenkins	http://localhost:30090	30090	contraseña inicial via kubectl exec
SonarQube	http://localhost:30092	30092	admin / admin
Prometheus	http://localhost:30093	30093	—
Grafana	http://localhost:30094	30094	admin / K8s Secret grafana-credentials
MCP K8s Server	http://mcp-k8s-server:3100 (interno)	ClusterIP	—

```
julia@JulianDc17 MINGW64 ~/OneDrive/MAS/Fundamentos Devops/U3/EFEFIC-FDVP20261-CI-CD-U3 (main)
$ kubectl get pods -n devops-lab -o wide
NAME                                READY   STATUS    RESTARTS   AGE    IP          NODE             NOMINATED NODE   READINESS GATES
devops-app-644fb7d78d-c4s6w         1/1     Running   0          16m    10.1.1.182  docker-desktop   <none>           <none>
devops-app-644fb7d78d-x4bvd         1/1     Running   0          16m    10.1.1.183  docker-desktop   <none>           <none>
grafana-c7ddfd78-84n5h              1/1     Running   0          171m   10.1.1.173  docker-desktop   <none>           <none>
jenkins-85888d694-vq6dl             1/1     Running   0          95m    10.1.1.177  docker-desktop   <none>           <none>
mcp-k8s-server-69c69cb6cd-svzgd     1/1     Running   0          3h5m   10.1.1.163  docker-desktop   <none>           <none>
prometheus-56cd666f4d-72whg         1/1     Running   0          3h5m   10.1.1.164  docker-desktop   <none>           <none>
sonarqube-5d4454bbdb-g4zrz          1/1     Running   0          3h3m   10.1.1.166  docker-desktop   <none>           <none>

julia@JulianDc17 MINGW64 ~/OneDrive/MAS/Fundamentos Devops/U3/EFEFIC-FDVP20261-CI-CD-U3 (main)
$
```

11. Resultados Obtenidos

Pipeline CI — GitHub Actions

Job	Estado	Detalle
1 — Maven Build & Test	PASSED	mvn compile + test (4/4 pruebas) + package. Artefacto .jar publicado como GitHub artifact.
2 — SonarQube	PASSED	Quality Gate: Passed. Cobertura JaCoCo reportada. Análisis disponible en sonarcloud.io.
3 — Snyk	PASSED	Escaneo de dependencias Maven completado. Resultados en GitHub Security → Code scanning.
4 — Docker Build & Push	PASSED	Imagen juliandc17/devops-app:latest y :sha-XXXXX publicadas en Docker Hub.
5 — MCP Health Check	PASSED	Verificación informativa del clúster. No bloquea el pipeline.

Pipeline CD — Jenkins

Stage	Estado	Evidencia
Inicio	PASSED	Workspace limpio, checkout de rama main exitoso.
Verificar Imagen	PASSED	docker pull juliandc17/devops-app:latest — imagen disponible en Docker Hub.
Preparar K8s	PASSED	Namespace devops-lab activo. kubectl cluster-info respondió correctamente.
Desplegar Aplicación	PASSED	Rolling update completado. 2/2 réplicas disponibles. kubectl rollout status OK.
MCP — Verificar	PASSED	get_pods retorna pods Running. rollout_status confirma éxito del deploy.
Smoke Test	PASSED	GET /actuator/health → {"status": "UP"} desde dentro del clúster.

Estado Final del Clúster

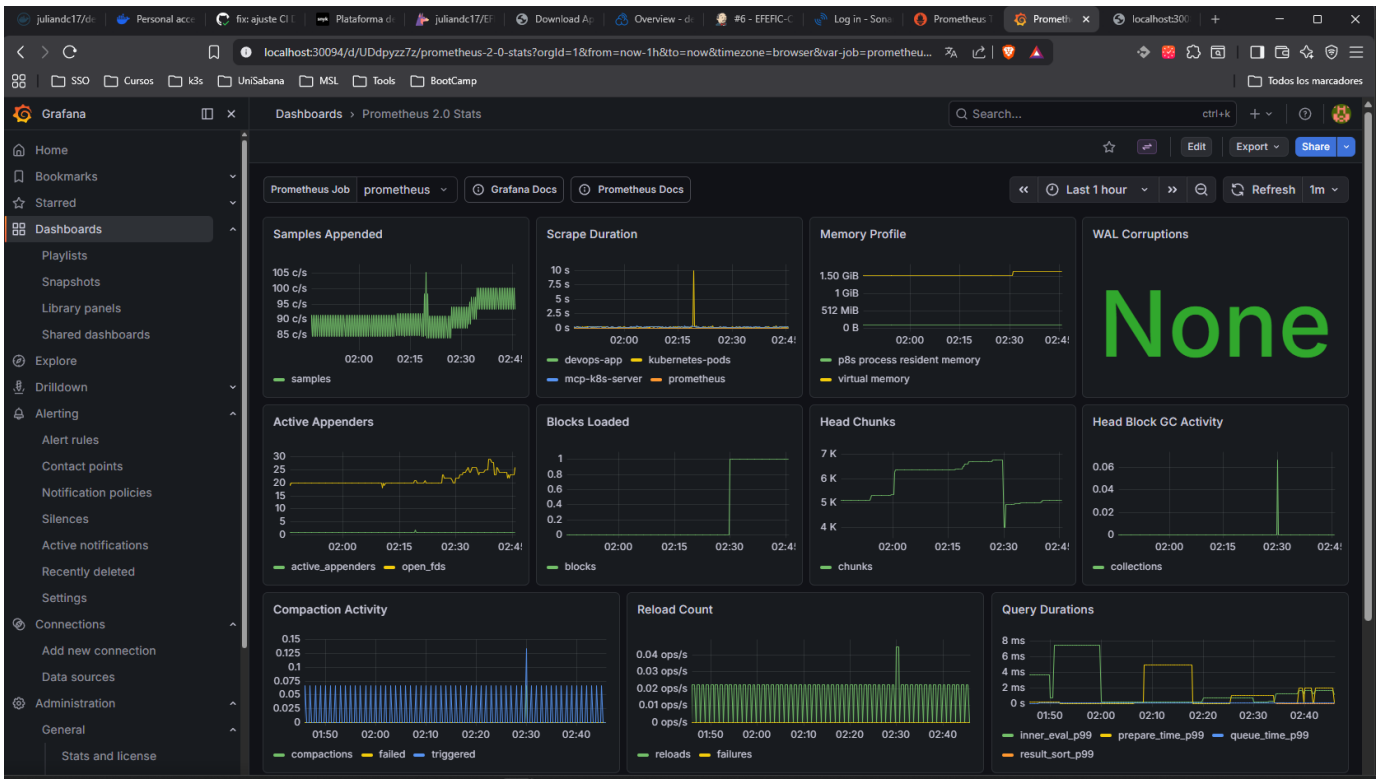
```
julia@JulianDc17 MINGW64 ~/OneDrive/MAS/Fundamentos Devops/U3/EFEFIC-FDVP20261-CI-CD-U3 (main)
$ kubectl get all -n devops-lab
NAME                                  READY   STATUS    RESTARTS   AGE
pod/devops-app-644fb7d78d-c4s6w       1/1     Running   0          17m
pod/devops-app-644fb7d78d-x4bvd       1/1     Running   0          16m
pod/grafana-c7ddfd78-84n5h            1/1     Running   0          172m
pod/jenkins-85888d694-vq6dl           1/1     Running   0          95m
pod/mcp-k8s-server-69c69cb6cd-svzgd   1/1     Running   0          3h6m
pod/prometheus-56cd666f4d-72whg       1/1     Running   0          3h6m
pod/sonarqube-5d4454bbdb-g4zrz        1/1     Running   0          3h4m

NAME                     TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)                        AGE
service/devops-app       NodePort    10.105.153.88    <none>        8080:30080/TCP                 3h6m
service/grafana          NodePort    10.103.250.61    <none>        3000:30094/TCP                 3h6m
service/jenkins          NodePort    10.110.179.129   <none>        8080:30090/TCP,50000:30091/TCP 125m
service/mcp-k8s-server   ClusterIP   10.105.98.148    <none>        3100/TCP                       3h6m
service/prometheus       NodePort    10.101.211.130   <none>        9090:30093/TCP                 3h6m
service/sonarqube        NodePort    10.96.91.254     <none>        9000:30092/TCP                 3h4m

NAME                              READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/devops-app        2/2     2            2           175m
deployment.apps/grafana           1/1     1            1           3h6m
deployment.apps/jenkins           1/1     1            1           125m
deployment.apps/mcp-k8s-server    1/1     1            1           3h6m
deployment.apps/prometheus        1/1     1            1           3h6m
deployment.apps/sonarqube         1/1     1            1           3h4m

NAME                                        DESIRED   CURRENT   READY   AGE
replicaset.apps/devops-app-5fc58b4bd4       0         0         0       75m
replicaset.apps/devops-app-644fb7d78d       2         2         2       17m
replicaset.apps/devops-app-6ffc4dddfd       0         0         0       175m
replicaset.apps/devops-app-77f666885d       0         0         0       94m
replicaset.apps/grafana-656cbfd86d          0         0         0       172m
replicaset.apps/grafana-79d4f875cb          0         0         0       3h6m
replicaset.apps/grafana-c7ddfd78            1         1         1       172m
replicaset.apps/jenkins-85888d694           1         1         1       95m
replicaset.apps/jenkins-8648d55fd6          0         0         0       125m
replicaset.apps/mcp-k8s-server-69c69cb6cd   1         1         1       3h6m
replicaset.apps/prometheus-56cd666f4d       1         1         1       3h6m
replicaset.apps/sonarqube-5d4454bbdb        1         1         1       3h4m

julia@JulianDc17 MINGW64 ~/OneDrive/MAS/Fundamentos Devops/U3/EFEFIC-FDVP20261-CI-CD-U3 (main)
$
```



12. Relación con Escenarios Reales y Principios DevOps

La arquitectura implementada en este laboratorio extiende el trabajo de la Unidad 2 con patrones avanzados que reflejan cómo organizaciones tecnológicas de escala mundial diseñan sus plataformas de entrega de software en la actualidad.

De CI/CD básico a DevSecOps

En la Unidad 2 se estableció la base del pipeline con GitHub Actions y Jenkins. La Unidad 3 eleva ese pipeline integrando SonarQube y Snyk directamente en el flujo de CI, siguiendo el principio de shift-left security: detectar vulnerabilidades lo más temprano posible en el ciclo de desarrollo, cuando corregirlas tiene el menor costo. Este enfoque es el mismo que aplican organizaciones como Google, Netflix y Amazon en sus pipelines de producción, donde la seguridad no es una fase al final sino una validación continua en cada commit.

Kubernetes como plataforma unificada

Una decisión arquitectural clave fue consolidar todas las herramientas dentro del mismo namespace de Kubernetes. Esto garantiza que todos los servicios se comuniquen por nombre DNS interno —por ejemplo, http://prometheus:9090 funciona desde cualquier pod— eliminando la complejidad de configurar redes entre plataformas distintas. Esta cohesión refleja cómo los equipos DevOps maduros diseñan su infraestructura de plataforma: todo declarativo, todo versionado, todo reproducible.

Observabilidad y convergencia con IA

Prometheus y Grafana implementan los tres pilares de la observabilidad: métricas (Prometheus scraping cada 15s), visualización (Grafana dashboards JVM y K8s) y alertas (reglas configuradas por YAML). El MCP K8s Server agrega una dimensión innovadora: permite que tanto el pipeline Jenkins como herramientas de IA como Claude Desktop consulten el estado del clúster en lenguaje natural. Esta convergencia entre DevOps e Inteligencia Artificial representa la dirección hacia donde evoluciona la industria.

Gestión de secretos y seguridad del repositorio

La gestión de credenciales a tres niveles —GitHub Secrets para CI, Jenkins Credentials para CD y Kubernetes Secrets para los pods— garantiza que ningún valor sensible aparezca en el repositorio público. Esta práctica previene las filtraciones de credenciales, que según múltiples reportes de la industria representan uno de los vectores de ataque más frecuentes en infraestructura cloud. El archivo .env.example en el repositorio documenta qué variables se necesitan sin exponer los valores reales, siguiendo el principio de 'documentar sin comprometer'.

Infraestructura como código

La aplicación, su pipeline CI, su pipeline CD, su infraestructura K8s, su configuración de seguridad y su stack de monitoreo están todos versionados, son reproducibles y auditables. Cualquier miembro del equipo puede recrear el entorno completo ejecutando los mismos comandos kubectl apply, garantizando paridad total entre entornos de desarrollo y producción. Esto elimina el problema histórico de 'funciona en mi máquina' que ha sido una fuente constante de fricción en los equipos de desarrollo.